2.1 环境准备与安装

> **生成模型**：OpenAI GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~25k tokens，输出 ~3.5k tokens（本节）

读完第 1 章之后，很多人最想做的事其实很简单：别再讲概念了，先让我把 OpenClaw 跑起来。这个想法完全对。理解一个系统，最好的办法往往不是先钻源码，而是先把它装上、点亮、发出第一条消息。你得先摸到它，后面读架构才不会飘。

这一节我们就做三件事：把运行环境准备好、选一种合适的安装方式、确认 `openclaw` 命令真的能用。整节都偏实操，尽量少讲大词，多讲你现在应该敲什么命令、看到什么结果、出了问题先查哪儿。

先说结论。对大多数第一次接触 OpenClaw 的读者，最省事的路径是：

1. 安装 Node.js 22+
2. 准备好 pnpm（如果你打算从源码跑）
3. 用 `npm install -g openclaw` 直接安装 CLI，或者从源码克隆后自己构建
4. 运行 `openclaw --help` 和 `openclaw doctor`

只要这几步走通，后面的引导向导、Gateway、第一条消息就都顺了。

2.1.1 你到底需要准备什么

OpenClaw 的入门门槛不算高，但也不是“装个二进制文件点两下”那种工具。它本质上是一个运行在本机上的 Node.js 应用，所以至少要准备下面这些东西：

- Node.js `22+`
- 一台可正常联网的电脑
- 终端环境：macOS Terminal / iTerm2、Linux shell，或者 Windows 上的 WSL
- 一个可用的模型提供方账号，后面向导里要配 API Key 或其他认证

如果你用的是 Windows，我还是建议你优先走 WSL2。原因很现实：本书后面会频繁碰到路径、权限、守护进程、日志、脚本这些东西。WSL 的使用体验跟 Linux/macOS 更接近，少踩很多“明明命令一样，为什么我这里就是不行”的坑。

先检查你机器上有没有 Node：

```
node --version
npm --version
```

理想情况下你会看到类似输出：

```
v22.14.0
10.9.2
```

如果 `node --version` 报错，说明还没装。如果能运行但主版本低于 22，也别抱着“应该差不多能跑”的心态硬试。OpenClaw 依赖较新的 Node 运行时能力，旧版本常见情况不是立刻崩，而是某些命令能执行、某些命令莫名其妙地不稳定，这种问题最烦。

2.1.2 安装 Node.js 22+

macOS / Linux：推荐用版本管理器

如果你平时会写前端、Node 工具，强烈建议装 `nvm` 、 `fnm` 或 `volta` 这类版本管理器。这样以后切 Node 版本很轻松，不会把系统环境弄乱。

最常见的是 `nvm` ：

```
nvm install 22
nvm use 22
node --version
```

预期输出大概像这样：

```
Now using node v22.14.0
v22.14.0
```

如果你还没装 `nvm` ，可以去它的官方仓库按说明安装。这里不展开，是因为不同 shell（bash、zsh、fish）配置略有区别。

Windows：推荐 WSL2 里安装

如果你在 Windows 上，建议先装 WSL2，再进入 Ubuntu 之类的发行版里安装 Node 22。这样你后面运行 OpenClaw 时，命令、目录和书里的示例基本一致。

进入 WSL 后，同样建议用 `nvm`：

```
nvm install 22
nvm use 22
node --version
```

不想折腾版本管理器怎么办

也可以直接去 Node.js 官网安装 22.x 的 LTS 版本，或者用 Homebrew / apt / winget 等系统包管理器安装。但要注意一件事：有些 Linux 发行版仓库里的 Node 版本偏旧，装完以后你以为自己“装上了”，实际只有 18 或 20。一定记得回头再查一次版本。

2.1.3 pnpm 要不要装

如果你只想把 OpenClaw 当成一个现成工具来用，直接全局安装 CLI，其实不一定需要你手动准备 pnpm。

但如果你打算：

- 从源码克隆 OpenClaw
- 自己构建
- 以后顺手改点代码、跑测试、看目录结构

那 pnpm 基本是必备的。OpenClaw 用的是 pnpm 工作区，依赖管理和脚本运行都围绕它展开。

最推荐的方式是用 Corepack。Corepack 是 Node 自带的包管理器代理，它能帮你对齐项目要求的 pnpm 版本。

```
corepack enable
pnpm --version
```

正常的话会看到：

```
10.23.0
```

如果你的系统提示 `corepack: command not found`，那通常有两种可能：

1. Node 装得不完整或版本太老
2. 当前 shell 还没拿到新版 Node 的环境

这时你可以先重新开一个终端再试。如果还是不行，也可以退一步直接安装：

```
npm install -g pnpm@10
pnpm --version
```

2.1.4 安装 OpenClaw：两条路怎么选

OpenClaw 常见的安装方式有两种。

第一种是直接装 CLI，适合“我先用起来再说”的读者；第二种是从源码构建，适合“我既想用，也想顺手看内部结构”的读者。

路线 A：直接全局安装 CLI

这是最快的方式：

```
npm install -g openclaw
```

安装结束后，先别急着继续，立刻检查命令有没有进 PATH：

```
openclaw --help
```

你应该会看到一串命令帮助，类似：

```
Usage: openclaw [options] [command]

Commands:
  onboard
  gateway
  agent
  doctor
  channels
  models
  help [command]
```

如果这里就报 `command not found: openclaw` ，通常不是 OpenClaw 本身坏了，而是全局 npm bin 目录没进 PATH。macOS 和 Linux 上常见原因是 shell 配置没刷新；Windows/WSL 上则可能是装在了另一个 Node 环境里。

这时候别乱猜，先跑：

```
npm config get prefix
```

看一下全局包到底装到了哪里，再检查那个目录是否在 PATH 里。

路线 B：从源码克隆并构建

如果你想后面边用边看仓库结构，这条路更合适。

```
git clone https://github.com/openclaw/openclaw.git
cd openclaw
pnpm install
pnpm build
```

这里的体感通常是： `pnpm install` 比较久， `pnpm build` 也不会像一个小玩具项目那样一闪而过。别慌，第一次构建慢一点很正常。

如果过程顺利，你大概率会看到类似日志：

```
Lockfile is up to date
Packages: +xxxx
...

Done in 32.4s

> openclaw@... build /path/to/openclaw
> ...

Build completed successfully
```

构建完以后，可以先验证 CLI 入口能不能工作：

```
node dist/entry.js --help
```

如果这个命令能打印帮助信息，说明构建基本没问题。

到底选哪条

如果你只是想跟着本章快速上手，优先选路线 A。省时间。

如果你已经确定后面要研究源码，或者你本来就习惯在仓库里跑命令，那就选路线 B。你会更容易把“书里说的东西”和“仓库里的真实结构”对起来。

2.1.5 从源码安装时，UI 要不要单独构建

这个问题初学者很容易纠结。简单说：看你要不要尽快接触控制台 UI。

如果你只是想先走通 CLI → Gateway → 模型这条最小链路，很多场景下 `pnpm build` 已经够你继续往下学了。

如果你想把控制台界面也准备好，可以再跑一次：

```
pnpm ui:build
```

有些同学会问，为什么 UI 还要单独构建？因为它不是“顺手拼在一起的静态页面”，而是一个独立前端包。你可以先把它理解成：OpenClaw 的核心服务和 Web 控制界面不是完全同一个编译目标，所以第一次从源码使用时，UI 相关资源可能需要单独准备。

2.1.6 安装完成后，先做健康检查

无论你走哪条安装路线，都建议立刻跑一次：

```
openclaw doctor
```

这是非常值得记住的命令。后面你遇到“命令能敲，但就是不返回”“Gateway 似乎起来了，但消息没发出去”“模型配置好像有问题”这类情况，第一反应都应该是先跑它。

你看到的输出可能会和你的环境有关，但大致会像这样：

```
[doctor] Checking runtime...
[doctor] Checking configuration...
[doctor] Checking gateway availability...
[doctor] Checking model setup...

Result: some checks need attention
```

如果你还没做首次配置， `doctor` 报一些提醒是正常的。现在它的意义不是要求你“全绿”，而是确认 CLI 至少能正常执行，环境检查链路是通的。

2.1.7 三个平台最容易踩的坑

macOS

macOS 通常是最省心的平台，但也有两个常见问题。

第一，Homebrew 安装的 Node 和你之前别的方式装的 Node 打架。你输入 `node --version` 看着是 22，实际 `npm install -g` 又装到另一个目录里，最后 `openclaw` 找不到。这个时候最重要的是把 Node 来源统一，别一会儿 `brew` ，一会儿 `nvm` ，一会儿系统自带。

第二，zsh 配置没刷新。你刚装完东西，当前窗口里 PATH 还是旧的，结果命令死活找不到。最简单的办法就是重开一个终端窗口再试。

Linux

Linux 最大的问题往往不是 OpenClaw，而是系统包版本太旧。尤其是一些稳定发行版，仓库里的 Node 可能跟书里要求差了两代。你用系统包管理器一装，命令是有了，版本却不对。解决思路也很直接：用版本管理器，别跟系统仓库硬耗。

另一个常见问题是缺少构建链路里依赖的基础工具，比如 `git` 、 `build-essential` 一类没装全。 `pnpm install` 或某些依赖 postinstall 阶段出错时，先别把锅扣到 OpenClaw 头上，先看系统工具是否完整。

Windows + WSL

WSL 最大的坑是路径混用。比如你人在 WSL 里，却把仓库放在 `/mnt/c/...` 下，再配合 Windows 侧编辑器和 Linux 侧命令一起用，性能和权限都可能有点别扭。更稳妥的方式是直接把仓库放在 WSL 的 Linux 文件系统里，比如 `~/projects/openclaw` 。

另一个坑是网络环境。你在 Windows 宿主机能访问模型服务，不代表 WSL 里也一定畅通。后面如果配 API Key 后模型调用失败，要记得把“WSL 这层网络是不是也能访问外部服务”纳入排查范围。

2.1.8 几个特别常见的报错，先别慌

`openclaw: command not found`

大概率是 PATH 问题，不是安装包损坏。先查全局 npm 安装目录，再看 PATH。

Node 版本不够

如果提示运行时版本太低，别试图绕过去。直接升级到 Node 22+，这是最省时间的修法。

`pnpm: command not found`

说明 pnpm 还没装，或者 Corepack 没启用。按前面的步骤补装就行。

`pnpm build` 很慢或者失败

第一次依赖安装和构建本来就可能慢。失败时先看错误发生在依赖下载、脚本执行，还是系统工具缺失。别只盯着最后一行红字，往上翻几十行通常更有用。

`openclaw doctor` 提示配置缺失

这反而说明前面的安装链路是通的，只是你还没做首次配置。下一节我们就处理这件事。

2.1.9 到这里，什么算“装好了”

很多人会把“我已经装好了”理解成“ `npm install` 没报错”。这其实不够。

对本章来说，更靠谱的完成标准是下面这三个：

```
node --version
openclaw --help
openclaw doctor
```

如果这三条都能正常执行，你就已经跨过了最麻烦的第一道坎。接下来我们要做的，不再是“让命令存在”，而是“让 OpenClaw 知道该用哪个模型、监听哪个 Gateway、后面准备连接哪个通道”。也就是下一节的主题：首次引导配置。

本节小结

- 运行 OpenClaw 的最低门槛不高，但 Node.js 版本必须到 `22+` ，这点别省。
- 只想快速体验，直接 `npm install -g openclaw` 最省事；想边用边看仓库，就走 `git clone` + `pnpm install` + `pnpm build` 。
- `pnpm` 主要是给源码安装和后续开发准备的，推荐用 Corepack 对齐版本。
- 安装完成后，别只看“命令装上了”，要用 `openclaw --help` 和 `openclaw doctor` 做一次真正的验证。

- macOS、Linux、Windows WSL 都能跑，但 Windows 读者优先用 WSL2，后面会顺很多。